Scenario-Based Coding Problems

Read passage → Identify data structure → Write solution

Elevator • Hospital • Bank • Parking • Library • Inventory • Call Center • Traffic

Each problem is a real-world passage. Read it carefully, identify the data structure, then write the code. This is exactly how iMocha and interview coding questions are framed.

PRIORITY QUEUE SCENARIOS

Scenario 1: Hospital Emergency Room [Priority Queue / Max-Heap]

A hospital emergency room receives patients throughout the day. Each patient is assigned a severity score when they arrive: 1 means minor injury, 5 means life-threatening. The doctor must always treat the most severely injured patient first, regardless of arrival order. If two patients have the same severity, the one who arrived first should be treated first. The system must support: adding a new patient, getting the next patient to treat, and checking if anyone is waiting. Design this patient queue system.

■ *Hint: Max-heap by severity. Use arrival time as tiebreaker. PriorityQueue with custom comparator in Java.*

Solution:

```
import java.util.*;

class Patient {
  String name;
  int severity;    // 1-5, higher = treat first
  int arrivalTime; // lower = arrived earlier

  Patient(String name, int severity, int arrivalTime) {
    this.name = name;
    this.severity = severity;
    this.arrivalTime = arrivalTime;
  }
}

class EmergencyRoom {
  // Max-heap: higher severity first, then earlier arrival
  PriorityQueue queue = new PriorityQueue<>((a, b) -> {
    if (b.severity != a.severity)
      return b.severity - a.severity;   // higher severity first
    return a.arrivalTime - b.arrivalTime; // earlier arrival first
  });

  int timer = 0;

  void addPatient(String name, int severity) {
    queue.offer(new Patient(name, severity, timer++));
    System.out.println(name + " added with severity " + severity);
  }

  void treatNext() {
    if (queue.isEmpty()) {
      System.out.println("No patients waiting");
      return;
    }
    Patient p = queue.poll();
    System.out.println("Treating: " + p.name + " (severity=" + p.severity + ")");
  }

  boolean hasPatients() { return !queue.isEmpty(); }
}

// Test
EmergencyRoom er = new EmergencyRoom();
er.addPatient("Ali",   2);
er.addPatient("Sara",  5);
er.addPatient("Ahmed", 5);
er.addPatient("Bilal", 1);
er.treatNext(); // Sara (severity 5, arrived before Ahmed)
er.treatNext(); // Ahmed (severity 5)
er.treatNext(); // Ali (severity 2)
er.treatNext(); // Bilal (severity 1)
```

✓ Key insight: Max-heap by severity. Tiebreaker = arrivalTime (min). Custom comparator handles both conditions.

Scenario 2: Smart Elevator System [Priority Queue / Greedy]

A smart elevator serves a 10-floor building. Requests arrive from different floors. The elevator is currently on floor 1 and moving UP. The system uses a priority queue where requests in the current direction of travel are processed first (lowest floor number while going up, highest floor number while going down). If no requests remain in the current direction, the elevator reverses. Requests: floor 3 going up, floor 7 going up, floor 2 going up, floor 8 going up. Starting at floor 1, going UP. Simulate the order floors are visited.

■ *Hint: Two priority queues: one for upward floors (min-heap), one for downward floors (max-heap). Process up queue first.*

Solution:

```
import java.util.*;

class Elevator {
  int currentFloor = 1;
  boolean goingUp = true;

  // Going up: process smallest floor first (min-heap)
  PriorityQueue upQueue   = new PriorityQueue<>();
  // Going down: process largest floor first (max-heap)
  PriorityQueue downQueue = new PriorityQueue<>(Collections.reverseOrder());

  void addRequest(int floor) {
    if (floor >= currentFloor)
      upQueue.offer(floor);
    else
      downQueue.offer(floor);
    System.out.println("Request added for floor " + floor);
  }

  void processAll() {
    while (!upQueue.isEmpty() || !downQueue.isEmpty()) {
      if (goingUp) {
        while (!upQueue.isEmpty()) {
          currentFloor = upQueue.poll();
          System.out.println("Going UP  -> Floor " + currentFloor);
        }
        goingUp = false; // reverse direction
      } else {
        while (!downQueue.isEmpty()) {
          currentFloor = downQueue.poll();
          System.out.println("Going DOWN -> Floor " + currentFloor);
        }
        goingUp = true;
      }
    }
  }
}

Elevator e = new Elevator();
e.addRequest(3); e.addRequest(7);
e.addRequest(2); e.addRequest(8);
e.processAll();
// Floor 2 -> 3 -> 7 -> 8 (all going up, no down requests)
```

✓ Key insight: Two heaps for two directions. Process all floors in current direction, then reverse. SCAN algorithm.

Scenario 3: Print Job Scheduler [Priority Queue + Simulation]

A printer receives multiple print jobs. Each job has a name, number of pages, and priority (1=low, 3=high). High priority jobs must print before low priority ones regardless of submission order. If priorities are equal, shorter jobs (fewer pages) print first to maximize throughput. The system must support: submit a job, print next job, see how many jobs are pending. Jobs submitted: Report(50 pages, priority 1), Invoice(5 pages, priority 3), Email(2 pages, priority 2), Contract(30 pages, priority 3). Write the scheduler.

■ *Hint: Max-heap by priority. Tiebreaker: min pages (shorter job first). PriorityQueue with 2-level comparator.*

Solution:

```
class PrintJob {
  String name;
  int pages;
  int priority; // 1=low, 3=high

  PrintJob(String name, int pages, int priority) {
    this.name = name; this.pages = pages; this.priority = priority;
  }
}

class PrintScheduler {
  PriorityQueue queue = new PriorityQueue<>((a, b) -> {
    if (b.priority != a.priority)
      return b.priority - a.priority; // higher priority first
    return a.pages - b.pages;         // fewer pages first (tiebreak)
  });

  void submit(String name, int pages, int priority) {
    queue.offer(new PrintJob(name, pages, priority));
  }

  void printNext() {
    if (queue.isEmpty()) { System.out.println("Queue empty"); return; }
    PrintJob job = queue.poll();
    System.out.println("Printing: " + job.name +
        " | Pages: " + job.pages + " | Priority: " + job.priority);
  }

  int pending() { return queue.size(); }
}

PrintScheduler ps = new PrintScheduler();
ps.submit("Report",   50, 1);
ps.submit("Invoice",   5, 3);
ps.submit("Email",     2, 2);
ps.submit("Contract", 30, 3);

while (ps.pending() > 0) ps.printNext();
// Invoice (priority 3, 5 pages)
// Contract (priority 3, 30 pages)
// Email (priority 2)
// Report (priority 1)
```

✓ Key insight: Two-level sort: priority DESC, then pages ASC. Lambda comparator chains both conditions cleanly.

LINKED LIST SCENARIOS

Scenario 4: Browser History (Back/Forward) [Doubly LinkedList / Stack]

A web browser maintains a browsing history. When a user visits a new page, it is added to the history. The user can press BACK to go to the previous page and FORWARD to go to the next page. If the user visits a new page after pressing BACK, all forward history is cleared (just like a real browser). The system must support: visit(url), back(), forward(), getCurrentPage(). Simulate: visit google, visit youtube, visit github, press back, press back, visit stackoverflow (forward history should clear), press back.

■ *Hint: Two stacks: backStack and forwardStack. Visit = push to back, clear forward. Back = pop from back, push to forward.*

Solution:

```
import java.util.Stack;

class BrowserHistory {
  Stack backStack    = new Stack<>();
  Stack forwardStack = new Stack<>();
  String current;

  BrowserHistory(String homepage) {
    current = homepage;
  }

  void visit(String url) {
    backStack.push(current);
    forwardStack.clear(); // clear forward on new visit
    current = url;
    System.out.println("Visited: " + url);
  }

  String back() {
    if (backStack.isEmpty()) return current;
    forwardStack.push(current);
    current = backStack.pop();
    System.out.println("Back to: " + current);
    return current;
  }

  String forward() {
    if (forwardStack.isEmpty()) return current;
    backStack.push(current);
    current = forwardStack.pop();
    System.out.println("Forward to: " + current);
    return current;
  }

  String getCurrent() { return current; }
}

BrowserHistory b = new BrowserHistory("home");
b.visit("google");
b.visit("youtube");
b.visit("github");
b.back();          // youtube
b.back();          // google
b.visit("stackoverflow"); // forward cleared
b.back();          // google (not youtube)
```

✓ Key insight: backStack = pages you came from. forwardStack = pages you went back from. New visit clears forward.

Scenario 5: Undo/Redo Text Editor [Stack]

A simple text editor supports typing characters and two special operations: UNDO (removes the last typed character) and REDO (re-types the last undone character). However, if the user types a new character after an UNDO, the redo history is cleared permanently. Given a sequence of operations: type 'H', type 'e', type 'l', type 'l', type 'o', UNDO, UNDO, type 'p', type '!', REDO (should do nothing). Print the final text.

■ *Hint: mainStack holds typed chars. undoStack holds undone chars. Type = push to main, clear undo. Undo = pop main, push to undo.*

Solution:

```
import java.util.Stack;

class TextEditor {
  Stack text    = new Stack<>();
  Stack undoStack = new Stack<>();

  void type(char c) {
    text.push(c);
    undoStack.clear(); // new input clears redo
  }

  void undo() {
    if (!text.isEmpty()) {
      undoStack.push(text.pop());
    }
  }

  void redo() {
    if (!undoStack.isEmpty()) {
      text.push(undoStack.pop());
    }
  }

  String getText() {
    StringBuilder sb = new StringBuilder();
    for (char c : text) sb.append(c);
    return sb.toString();
  }
}

TextEditor e = new TextEditor();
e.type('H'); e.type('e'); e.type('l');
e.type('l'); e.type('o');
e.undo(); e.undo();   // removes 'o' and 'l'
e.type('p'); e.type('!'); // clears redo
e.redo(); // does nothing (undo stack cleared)
System.out.println(e.getText()); // "Help!"
```

✓ Key insight: Same pattern as browser history. Two stacks simulate undo/redo. New action always clears redo stack.

QUEUE / SIMULATION SCENARIOS

Scenario 6: Bank Teller Queue [Queue + Simulation]

A bank has 3 teller windows. Customers arrive and join the shortest queue. Each customer has a service time (in minutes). Simulate processing: Customer 1 (3 min), Customer 2 (5 min), Customer 3 (2 min), Customer 4 (4 min), Customer 5 (1 min), Customer 6 (6 min). Assign each customer to the teller with the least total remaining service time. Print which teller each customer goes to and total time each teller works.

■ *Hint: Use a min-heap of (current_load, teller_id). Each new customer goes to teller with minimum load.*

Solution:

```
import java.util.*;

class BankSimulator {
  // Min-heap: [totalTime, tellerId]
  PriorityQueue tellers = new PriorityQueue<>(
    (a, b) -> a[0] - b[0]
  );

  BankSimulator(int numTellers) {
    for (int i = 1; i <= numTellers; i++)
      tellers.offer(new int[]{0, i}); // load=0, id=i
  }

  void assignCustomer(int customerId, int serviceTime) {
    int[] teller = tellers.poll(); // least loaded
    System.out.println("Customer " + customerId +
        " -> Teller " + teller[1] +
        " (current load: " + teller[0] + " min)");
    teller[0] += serviceTime; // update load
    tellers.offer(teller);    // put back
  }

  void printFinalLoads() {
    System.out.println("\nFinal workload:");
    List list = new ArrayList<>(tellers);
    list.sort((a,b)->a[1]-b[1]);
    for (int[] t : list)
      System.out.println("Teller " + t[1] + ": " + t[0] + " min");
  }
}

BankSimulator bank = new BankSimulator(3);
int[] times = {3, 5, 2, 4, 1, 6};
for (int i = 0; i < times.length; i++)
  bank.assignCustomer(i+1, times[i]);
bank.printFinalLoads();
```

✓ **Key insight: Min-heap of teller loads. Always assign to minimum load teller. Classic 'task scheduler' pattern.**

Scenario 7: Call Center Agent Routing [Queue + Multiple Levels]

A call center has three levels of support: Level 1 (basic), Level 2 (technical), Level 3 (specialist). Incoming calls are always first assigned to a free Level 1 agent. If no Level 1 agent is free, the call waits in a queue. When a call is too complex for Level 1, it gets escalated to Level 2. If still unresolved, it goes to Level 3. Each level has 2 agents. Model: 5 agents total (2 per level). Track which agent handles each call. If all agents at a level are busy, the call waits. Write a dispatch system.

■ *Hint: Each level has a queue of free agents. Incoming calls go to first available agent. Escalation moves to next level queue.*

Solution:

```
import java.util.*;

class CallCenter {
  Queue[] levels = new Queue[3];

  CallCenter() {
    for (int i = 0; i < 3; i++) {
      levels[i] = new LinkedList<>();
    }
    // Add agents
    levels[0].add("L1-Agent1"); levels[0].add("L1-Agent2");
    levels[1].add("L2-Agent1"); levels[1].add("L2-Agent2");
    levels[2].add("L3-Agent1"); levels[2].add("L3-Agent2");
  }

  String dispatch(int callId) {
    for (int lvl = 0; lvl < 3; lvl++) {
      if (!levels[lvl].isEmpty()) {
        String agent = levels[lvl].poll();
        System.out.println("Call " + callId +
            " -> " + agent + " (Level " + (lvl+1) + ")");
        return agent;
      }
    }
    System.out.println("Call " + callId + " -> All agents busy, waiting");
    return null;
  }

  void freeAgent(String agent, int level) {
    levels[level].offer(agent); // agent becomes free again
  }
}

CallCenter cc = new CallCenter();
cc.dispatch(101);  // L1-Agent1
cc.dispatch(102);  // L1-Agent2
cc.dispatch(103);  // L2-Agent1 (L1 full)
cc.dispatch(104);  // L2-Agent2
cc.dispatch(105);  // L3-Agent1
cc.dispatch(106);  // L3-Agent2
cc.dispatch(107);  // All busy, waiting
```

✓ Key insight: Queue per level. Try each level in order. First available agent picks up. freeAgent() returns agent to pool.

HASHMAP / LOGIC SCENARIOS

Scenario 8: Library Book Management [HashMap + Nested Data]

A library system manages books and members. Each book has an ISBN, title, and availability status. Members can borrow and return books. The system must: add a book, borrow a book (mark unavailable, record who has it), return a book (mark available), check if a book is available, and show all books a specific member currently has. A member cannot borrow the same book twice. If a book is unavailable, borrowing should fail with a clear message.

■ *Hint: HashMap for O(1) lookup. HashMap> to track member borrowings.*

Solution:

```
import java.util.*;

class Book {
  String isbn, title;
  boolean available;
  String borrowedBy; // null if available

  Book(String isbn, String title) {
    this.isbn = isbn; this.title = title; this.available = true;
  }
}

class Library {
  Map       catalog     = new HashMap<>();
  Map> memberBooks = new HashMap<>();

  void addBook(String isbn, String title) {
    catalog.put(isbn, new Book(isbn, title));
  }

  boolean borrow(String memberId, String isbn) {
    Book book = catalog.get(isbn);
    if (book == null)           { System.out.println("Book not found"); return false; }
    if (!book.available)        { System.out.println(book.title + " not available, borrowed by: " + book.borrowedB
y); return false; }

    book.available  = false;
    book.borrowedBy = memberId;
    memberBooks.computeIfAbsent(memberId, k -> new ArrayList<>()).add(isbn);
    System.out.println(memberId + " borrowed: " + book.title);
    return true;
  }

  boolean returnBook(String memberId, String isbn) {
    Book book = catalog.get(isbn);
    if (book == null || book.available) { System.out.println("Invalid return"); return false; }

    book.available  = true;
    book.borrowedBy = null;
    memberBooks.getOrDefault(memberId, new ArrayList<>()).remove(isbn);
    System.out.println(memberId + " returned: " + book.title);
    return true;
  }

  void showMemberBooks(String memberId) {
    List books = memberBooks.getOrDefault(memberId, new ArrayList<>());
    System.out.println(memberId + " has: " + books.size() + " book(s)");
    for (String isbn : books)
      System.out.println("  - " + catalog.get(isbn).title);
  }
}

Library lib = new Library();
lib.addBook("978-1", "Clean Code");
lib.addBook("978-2", "Design Patterns");
lib.borrow("M001", "978-1");
lib.borrow("M002", "978-1"); // fails - unavailable
lib.borrow("M002", "978-2");
lib.showMemberBooks("M001");
lib.returnBook("M001", "978-1");
lib.borrow("M002", "978-1"); // now works
```

✓ Key insight: computeIfAbsent creates list if key absent. getOrDefault returns empty list if member has no books.

Scenario 9: Parking Lot System [HashMap + Counting]

A multi-level parking lot has 3 levels with 10 spots each. Spots are numbered 1-10 per level. Small cars can park in any spot. Large cars can only park in spots 6-10 (the bigger spots). Motorcycles can park in any spot and take half a spot (2 motorcycles per spot). The system must: park a vehicle, find the nearest available spot for its type, remove a vehicle when it leaves, and show occupancy per level. Spot 1 is closest (preferred). Track by vehicle plate number.

■ *Hint: 2D array or Map>. Small=any spot, Large=spots 6-10, Moto=pairs share one spot.*

Solution:

```
import java.util.*;

class ParkingLot {
  // level -> spot -> plate (null=empty)
  Map> lot = new HashMap<>();
  // For motorcycles: spot -> count (max 2)
  Map motoCount = new HashMap<>();

  ParkingLot(int levels, int spotsPerLevel) {
    for (int l = 1; l <= levels; l++) {
      lot.put(l, new HashMap<>());
    }
  }

  boolean park(String plate, String type) {
    for (int level = 1; level <= lot.size(); level++) {
      Map floor = lot.get(level);
      int start = type.equals("LARGE") ? 6 : 1;

      for (int spot = start; spot <= 10; spot++) {
        String key = level + "-" + spot;
        if (type.equals("MOTO")) {
          int count = motoCount.getOrDefault(key, 0);
          if (count < 2) {
            motoCount.put(key, count + 1);
            floor.put(spot, (count==0 ? plate : floor.get(spot)+","+plate));
            System.out.println(plate + " parked at L"+level+"-S"+spot);
            return true;
          }
        } else if (!floor.containsKey(spot)) {
          floor.put(spot, plate);
          System.out.println(plate+" parked at Level "+level+" Spot "+spot);
          return true;
        }
      }
    }
    System.out.println("No spot available for " + type);
    return false;
  }

  void leave(String plate) {
    for (int level = 1; level <= lot.size(); level++) {
      Map floor = lot.get(level);
      for (int spot : floor.keySet()) {
        if (floor.get(spot) != null && floor.get(spot).contains(plate)) {
          floor.remove(spot);
          System.out.println(plate + " left from L"+level+"-S"+spot);
          return;
        }
      }
    }
  }
}

ParkingLot pl = new ParkingLot(3, 10);
pl.park("ABC-001", "SMALL");
pl.park("XYZ-002", "LARGE");
pl.park("MOT-001", "MOTO");
pl.park("MOT-002", "MOTO"); // shares spot with MOT-001
pl.leave("ABC-001");
```

✓ **Key insight: Nested maps for O(1) lookup. Iterate from spot 1 upward for nearest available. Motorcycles tracked separately.**

Scenario 10: Inventory Restock Alert System [HashMap + Priority Queue]

A warehouse tracks inventory for hundreds of products. Each product has a current stock level and a minimum threshold. When stock falls below the threshold, the product is added to a restock priority queue. Products with the lowest stock (closest to zero) are restocked first. The system must: update stock (decrease when sold), add to restock queue automatically when threshold breached, process restock orders from highest urgency to lowest, and show all products currently needing restock. Multiple sales can happen before restocking.

■ *Hint: HashMap for stock levels. Min-heap PQ for restock urgency (lowest stock = most urgent). Auto-trigger on threshold breach.*

Solution:

```
import java.util.*;

class Product {
  String id;
  int stock;
  int threshold;

  Product(String id, int stock, int threshold) {
    this.id = id; this.stock = stock; this.threshold = threshold;
  }
}

class Inventory {
  Map products = new HashMap<>();
  // Min-heap: lowest stock = most urgent restock
  PriorityQueue restockQueue = new PriorityQueue<>(
    (a, b) -> a.stock - b.stock
  );
  Set inRestockQueue = new HashSet<>(); // avoid duplicates

  void addProduct(String id, int stock, int threshold) {
    products.put(id, new Product(id, stock, threshold));
  }

  void sell(String id, int qty) {
    Product p = products.get(id);
    if (p == null) { System.out.println("Product not found"); return; }
    p.stock = Math.max(0, p.stock - qty);
    System.out.println(id + " stock: " + p.stock);

    // Auto-add to restock queue if below threshold
    if (p.stock < p.threshold && !inRestockQueue.contains(id)) {
      restockQueue.offer(p);
      inRestockQueue.add(id);
      System.out.println("  ■ RESTOCK ALERT: " + id);
    }
  }

  void processRestock(int qty) {
    if (restockQueue.isEmpty()) {
      System.out.println("Nothing to restock"); return;
    }
    Product p = restockQueue.poll();
    inRestockQueue.remove(p.id);
    p.stock += qty;
    System.out.println("Restocked " + p.id + " -> stock now: " + p.stock);
  }

  void showRestockList() {
    System.out.println("Needs restock: " + inRestockQueue);
  }
}

Inventory inv = new Inventory();
inv.addProduct("ITEM-A", 50, 20);
inv.addProduct("ITEM-B", 30, 25);
inv.addProduct("ITEM-C", 100, 10);
inv.sell("ITEM-A", 40); // stock=10, below 20 -> alert
inv.sell("ITEM-B", 20); // stock=10, below 25 -> alert
inv.sell("ITEM-C", 95); // stock=5,  below 10 -> alert
inv.showRestockList();
inv.processRestock(100); // restocks most urgent (ITEM-C: stock=5)
inv.processRestock(100); // next (ITEM-A or ITEM-B: both at 10)
```

✓ Key insight: HashSet prevents duplicate restock entries. Min-heap auto-prioritizes by stock level. sell() triggers alert automatically.

HOW TO READ A SCENARIO — 5-STEP APPROACH

Step 1: Identify the ENTITIES	What are the objects? Patient, Book, Car, Call. These become your classes.

Step 2: Identify the OPERATIONS	What actions happen? Add, remove, find, update. These become your methods.
Step 3: Identify the ORDER/PRIORITY	Does sequence matter? FIFO=Queue, Priority=PQ, LIFO=Stack, Fast lookup=HashMap
Step 4: Identify the RELATIONSHIPS	One-to-many? Use List. Key-value? Use HashMap. Sorted? Use TreeMap.
Step 5: Start with the data structure	Write the class and main data structure first. Then add methods one by one.

Keyword → Data Structure Mapping

Keyword in scenario	Data Structure to use
"process in order received", "queue", "first come first served"	Queue (FIFO)
"highest priority first", "most urgent", "most severe"	Priority Queue / Max-Heap
"lowest first", "minimum", "closest to zero"	Priority Queue / Min-Heap
"undo", "back button", "last action", "reverse order"	Stack (LIFO)
"look up by ID", "fast search", "key-value", "track who has"	HashMap
"sorted order", "alphabetical", "in order"	TreeMap or sorted list
"unique values", "no duplicates", "already exists check"	HashSet
"hierarchical", "parent-child", "file system"	Tree
"next/previous", "insert in middle", "chain"	LinkedList
"nearest available", "assign to least loaded"	Priority Queue (load balancing)